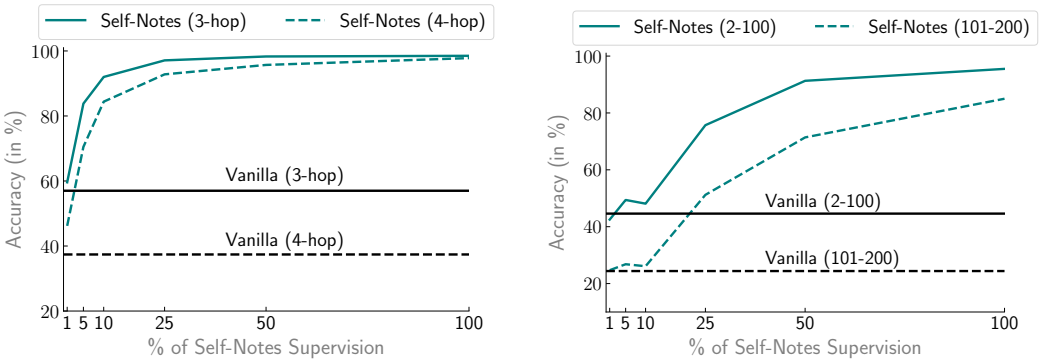

(a) Toy-Story

(b) Algorithmic

Figure 2: Performance of the Semi-supervised Self-Notes method with varying amounts of Self-Notes supervision on the Toy-Story and Algorithmic tasks.

Table 3: Unsupervised Self-Notes: Toy-Story task accuracy (%)

Method	**3-hop**	**4-hop**
Vanilla	79.4	57.9
+ Self-Notes (unsupervised)	79.7	61.8
+ Boost Questions	82.4	68.2
+ Multi-sample	91.3	79.1
+ Finetune	94.2	85.8

4.2 Semi-supervised Self-Notes

Figure 2 shows the performance of the Self-Notes method with varying amounts of Self-Note supervision for the Toy-Story and the Algorithmic tasks. That is, we randomly sample some percentage of the training samples that get Self-Note supervision. For Toy-Story, we find that even Self-Note supervision using as little as 1% of the training set (100 samples), leads to performance gains over the Vanilla model, and the performance starts to saturate around 25% supervision. On the other hand, for the Algorithmic task, we observe gains with Self-Note supervision starting at around 5% supervision, and the performance steadily improves with more Self-Note supervision.

4.3 Unsupervised Self-Notes

In our third set of experiments, we apply Self-Notes to a 1-variable Algorithmic task and Toy-Story task when we have no ground-truth Self-Notes to train on.

First, we conduct experiments in the unsupervised setting for Algorithmic task. We first test on a 1-variable version of the dataset (each sample contains only 1 variable). We train on datasets that contain varying length samples (i.e. varying numbers of algorithmic statements per sample), so the model will generate intermediate Self-Notes on its own in a QA form. In this task, we allow the model to generate Self-Notes, and then conditioning on the previous note to predict the next Self-Note and final answer during training, departing from the standard parallel training procedure. The model therefore has to do two simultaneous tasks during training, write the correct Self-Notes, and predict the final answer given the written Self-Notes. Since we only use 1-variable samples, it makes it straightforward to learn which Self-Note questions to write (it will always be `print x`, where `x` is the variable in that sample. We can see from Figure 3, that around 10k samples, the unsupervised Self-Notes method starts to learn how to write and leverage Self-Notes that improve accuracy over the Vanilla method. With 20k samples, the unsupervised Self-Notes method achieves near 100% accuracy, with a significant increase over the Vanilla model.

The second task we consider for unsupervised Self-Notes is Toy-Story. Here, the training data has 100k samples with 1 and 2 hop questions, but contains no Self-Notes. This task is more difficult since there are many more variables (people, objects, locations) and model needs to ask the right questions in Self-Notes.

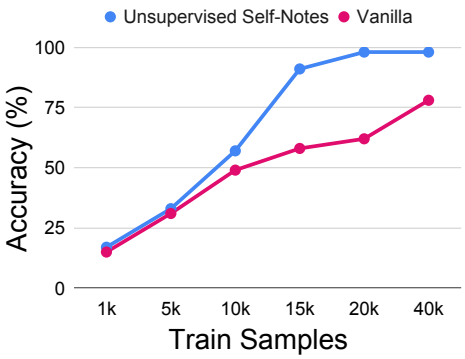

Figure 3: Unsupervised Self-Notes vs Vanilla sample comparison on 1-variable Algorithmic.

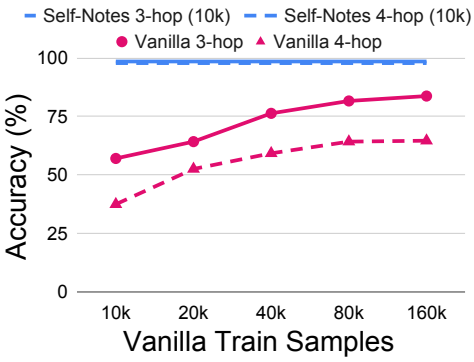

Figure 4: 10k Supervised Self-Notes samples vs Vanilla sample comparison on Toy-story.

We first train a Vanilla model to generate the final question and answer, with test accuracy shown at the top of Table 3.[3] Next, we test the vanilla model with Self-Notes by allowing it to generate QAs as notes. Here, we only add the answer parts to the context because the model has never seen a question in the context during training. Additionally, because the model is trained on 1-hop questions, it often asks a question whose answer is already in the context. We ignore such duplicate answers and move on to the next context sentence.

Simply adding Self-Notes to the Vanilla model during testing does not improve the performance much because the model is not trained to ask the right questions. We therefore encourage the model to ask more questions by boosting the probability of the question start token "Q:". Boosting Self-Notes by B=5 does improve the performance over the Vanilla model. Furthermore, generating multiple different Self-Notes by sampling questions and selecting the most confident one also helps. This is likely because when the right question is asked and answered, the model becomes more confident in its final answer. Finally, finetuning the model on a Self-Note version of the original training data improves the performance, as it adapts to longer stories. In summary, we see a significant increase in accuracy over the Vanilla results by allowing the model to generate Self-Notes and finetuning the model on the generations.

4.4 Few-shot Prompted Self-Notes

For the MultiArith task, we test a few-shot prompted GPT-J model [23]. For the Vanilla and Scratchpad baselines, we use the same few-shot prompt demonstrations from [4]. For Self-Notes, we use the same few-shot examples, but additionally annotated with Self-Notes that we wrote (provided in the Appendix). In Table 4, we compare Self-Notes to the Vanilla and Chain-of-Thought (CoT) methods for similar sized models (6.7B-8B). We find that the Self-Notes method on the 6B GPT-J model outperforms all of the Vanilla and CoT results. This result highlights two key findings. First, Self-Notes can write pertinent, yet mathematically accurate notes on arithmetic word problems. Second, we can effectively use Self-Notes on pretrained LLMs using few-shot prompting.

For the more challenging GSM8K task, we employed a GPT-3 model with 175B parameters. As shown in Table 5, Self-Notes outperforms CoT, but by a small margin. Since we prompted the model using the public API and had no access to token output probabilities over the prompt itself, we limit Self-Notes positions to immediately after ".", ",", or "?" to reduce the API calls. See [4] for more comprehensive comparison of CoT against Vanilla prompting (note that they used a more powerful version of GPT-3 than us).

Lastly, we run few-shot prompting experiments using the state-of-the-art open model, Llama 2 (70B) [24]. We compare Vanilla, Chain-of-Thought, and Self-Notes across three datasets: Algorithmic, MultiArith, and GSM8K. For GSM8K and MultiArith, we use the 8-shot arithmetic reasoning prompt from [4]. The CoT and Vanilla prompts are the same as from [4] and the Self-Notes prompt adapts the CoT prompt by putting some of the reasoning steps inside the context. For Algorithmic, we use a 5-shot prompt of randomly selected examples from our training set, ranging from 6-33 statements.

[3]Here we see better Vanilla performance than Table 2 because the training data is larger.

Table 4: GPT-3, LaMDA, PaLM, and GPT-J few-shot prompting accuracy (%) on MultiArith.

Model	Size	Vanilla	CoT	Self-Notes
GPT-3 [4]	6.7B	4.5	2.8	-
LaMDA [4]	8B	5.8	1.5	-
PaLM [4]	8B	4.2	15.8	-
GPT-J	6B	6.0	13.5	**21.0**

Table 5: GPT-3 few-shot prompting accuracy (%) on GSM8K.

Model	Size	CoT	Self-Notes
GPT-3	175B	11.4	**13.4**

Table 6: Llama 2 (70B) few-shot prompting accuracy (%).

Dataset	Vanilla	CoT	Self-Notes
Algorithmic	23.3	37.4	**40.3**
MultiArith	33.3	94.5	**96.2**
GSM8K	16.5	55.9	**59.9**

We test on the 2-100 statement test set from our paper. Results are shown in Table 6. In summary, we find that Self-Notes outperforms Chain-of-Thought and the Vanilla baseline on all three tasks.

4.5 Ablation: Labeled training set size comparison

Each Self-Notes training sample in Toy-Story has intermediate questions and answers, therefore increases the total number of QA pairs in the training set. For example if the final question and answer is "`Where is the ball? The ball is in the park`", but there was a Self-Note in the middle of the context labeled "`Who has the ball? Alice has the ball`", then that sample has two QA pairs. We therefore also run a comparison of the total number of labelled QA pairs between Self-Notes and Vanilla. Specifically, the 10k Self-Notes training data for Toy-Story has 10k total samples, and therefore 10k final QA pairs. However, it also includes roughly 70k Self-Note QA pairs which means that the total amount QA pairs is around 80k. Figure 4 shows the effect of increasing the training size for the Vanilla baseline compared to a fixed set of 10k Self-Notes training samples (80k labeled QA pairs) in the Toy-Story task. The Self-Notes model with 10k samples still vastly outperforms the Vanilla model with a 1500% increase in training samples (and roughly a 100% increase in the amount of QA pairs to that of Self-Notes). See Appendix for more ablation experiments, including understanding the difference between content and compute in Self-Notes by using "dummy" tokens (Appendix subsection A.3).

5 Related Work

Implicit Reasoning. bAbI [22] was a set of synthetic tasks for testing different reasoning capabilities [25] and showed the advantage of attention-based models over recurrent neural networks [26, 27]. More recently, attention-based transformers [1] have become the foundation of language-based reasoning [28]. However, the feedforward nature of transformers makes it unsuitable for state-tracking [11] and several recurrent versions have been proposed [29, 13, 14]. Further, transformer-based large LMs are shown to struggle at multi-step reasoning [7].

Explicit Rationales. Use of rationales has been explored for interpretability [30], and for performing intermediate computations [31, 3, 4]. In particular, the Scratchpad method by Nye et al. [3] is closest to our proposed Self-Notes method which can be interpreted as an *online*-variant of Scratchpad. Use of rationales for reasoning and arithmetic tasks, referred to as "chain-of-thought", has been shown to be particularly beneficial for zero- and few-shot in-context learning with large language models [4, 32, 7]. Zelikman et al. [9] showed the possibility of bootstrapping from a small set of reasoning labels to a larger unlabeled dataset. Trivedi et al. [33] propose interleaving chain-of-thought reasoning with knowledge retrieval steps (both after context). Other works have generated "inner monologue" tokens as a form of intermediate reasoning after a prompt [10, 34]. However, as with Scratchpad, the "chain-of-thought" and "inner monologue" reasoning is done after reading the entire input context rather than while reading it as in Self-Notes. This emergent capability of chain-of-thought prompting is only possible in large ($>$ 6B) models [4]. In concurrent work, Toolformer [35]